nwergm: Exponential-family random graph models in Stata

Thomas U. Grund
Institute of Sociology, RWTH Aachen University, Germany

Abstract. Exponential-family random graph models (ERGMs) are a standard tool for the statistical analysis of network data, but Stata users have had to fit them with other software. `nwergm` fits ERGMs in Stata. It uses the same estimation algorithms, control defaults and terms as the `ergm` package of the statnet project, entirely within Stata, and implements 123 of the 139 terms of `ergm` 4.12.0 fully and two in part. It handles directed, undirected and two-mode networks, curved and operator terms, and sampling and observation constraints; it estimates by maximum pseudolikelihood, Monte Carlo maximum likelihood, stochastic approximation or contrastive divergence, and offers Monte Carlo diagnostics, goodness of fit, information criteria, prediction and simulation. The sampler and the change statistics run in a compiled plugin. On the examples of the article the estimates agree with those of `ergm` to within 0.09 standard errors; on eight benchmark models a fit is faster by a factor of 2.1 to 4.1. The article describes the syntax, the term library, the estimation and the diagnostics, with an advice network among lawyers as the main worked example.

Keywords: st0003, nwergm, exponential-family random graph models, ERGM, network statistics, Monte Carlo maximum likelihood, Stata plugins, nwcommands

1 Introduction

An exponential-family random graph model (ERGM) treats a network's ties as the joint outcome of a small set of structural and covariate-driven tendencies, such as density, reciprocity, homophily, triadic closure and degree heterogeneity, which are estimated together; Lusher et al. (2013) give the standard treatment of the theory, estimation, and interpretation involved. The `ergm` package of the statnet project (Handcock et al., 2008; Hunter et al., 2008b; Krivitsky et al., 2023) has been the standard implementation for close to two decades. A Stata user who wanted to fit an ERGM had to leave Stata, export the network, fit the model in other software, such as `ergm` in R or PNet (Wang et al., 2009), and import the results back.

`nwergm` fits ERGMs in Stata. It is the ERGM command of `nwcommands` (Grund, 2026a), a family of Stata packages for network analysis, and works on any network declared with `nwset` or imported with `nwcommands`. It implements nearly all terms of `ergm` 4.12.0 (Table 1), its four estimators, its constraints and its treatment of missing ties, and Stata versions of its diagnostics, goodness of fit, simulation and prediction. When this article says that `nwergm` does something as `ergm` does, it means `ergm` 4.12.0.

Provenance. `nwergm` was developed from `ergm`: the term definitions and change statistics, the pseudolikelihood and Monte Carlo estimators, the control defaults and the diagnostics follow the R package and its published description (Hunter and Handcock,

2006; Hunter, 2007; Morris et al., 2008; Hummel et al., 2012; Geyer and Thompson, 1992). `nwergm` is not affiliated with or endorsed by the statnet project, and errors are mine. Where `ergm` and `nwergm` differ, Section 10 says so.

2 Installation

`nwergm` is a package of its own. It uses the network objects and data handling of `nwcommands` (Grund, 2026a), which must be installed first; installation is repeated here in full so this article stands on its own. Stata 17 or later is needed (the software is tested with Stata 17, 18 and 19; StataBE is enough). From within Stata:

```
. net install nwcommands, from(https://nwcommands.org/net/) replace
. nwinstall, everything
```

The first command installs `nwinstall`. The second installs the `nwcommands` core, the four model packages including `nwergm` and a "Network Analysis" menu, and prints the installed versions. Repeating the two lines updates the installation. The compiled plugin for macOS, Linux or Windows comes with the package, so no compiler is needed.

`help nwergm` opens the documentation, and `db nwergm` opens a dialog for every term and option. The manual (Grund, 2026c) is the technical reference; it is archived at Zenodo, `https://doi.org/10.5281/zenodo.23661331`. The source code is at `https://github.com/thomas-u-grund/nwergm`.

Versions

The logs in this article were produced with `nwergm` 1.5.7 (9 November 2026) and `nwcommands` 3.17.2 in Stata 18. The comparisons and timings of sections 9 and 9.3 were made with `nwergm` 1.5.4, whose estimation code is the same. Releases are numbered major.minor.patch: a change in the first number may change results or syntax, the second adds features, and the third fixes errors. The website explains how to install a particular release: `https://nwcommands.org/nwcommands/install/`.

`nwergm version` reports the installed release, and `e(version)` stores it after each fit, so that an analysis can state which release produced it. The do-file that produces every log of this article, `gen_logs.do`, is part of the supplemental materials.

License

`nwergm` is free software under the GNU General Public License, version 3, as are the `nwcommands` core and the other model packages. `nwergm` is derived from `ergm` (Handcock et al., 2008), which is distributed under the same license. Further worked tutorials are at `https://nwcommands.org/nwergm/`.

3 ERGM framework

An ERGM models an entire network y as one draw from an exponential-family distribution over the set $\mathcal{Y}$ of all possible networks on a fixed set of nodes:

$$P_\theta(Y = y) = \frac{\exp(\theta^\top g(y))}{\kappa(\theta)}, \qquad y \in \mathcal{Y}, \tag{1}$$

where $g(y)$ is a vector of network statistics (density, reciprocity, triadic closure and so on), θ is the corresponding vector of parameters, and $\kappa(\theta) = \sum_{y' \in \mathcal{Y}} \exp(\theta^\top g(y'))$ normalizes the distribution over every network on the same node set: a sum with $2^{n(n-1)}$ terms for a directed network on n nodes, intractable at any realistic size. In a curved model, θ is a function of a smaller parameter vector, as for the geometrically weighted terms whose decay is estimated (Hunter and Handcock, 2006).

Every term enters the model through its *change statistic*: how much the term's contribution to $g(y)$ changes if a single tie y_{ij} is toggled, holding every other tie fixed,

$$\Delta g_{ij}(y) = g(y_{ij}^{+}) - g(y_{ij}^{-}), \tag{2}$$

where y_{ij}^{+} and y_{ij}^{-} denote y with the ij tie forced present and absent. The conditional log odds of a single tie reduce to a linear form in the change statistic regardless of how complicated $g(y)$ is:

$$\log \frac{P_\theta(Y_{ij} = 1 \mid Y_{-ij} = y_{-ij})}{P_\theta(Y_{ij} = 0 \mid Y_{-ij} = y_{-ij})} = \theta^\top \Delta g_{ij}(y). \tag{3}$$

So a coefficient is the change in the conditional log odds of a tie per unit of the statistic, given the rest of the network. The pseudolikelihood uses this form: it is the likelihood of a logistic regression of the ties on their change statistics, which play the role of the covariates of dyad ij.

Whether the pseudolikelihood is also the likelihood depends on whether the terms are dyad-independent or dyad-dependent. If every term is dyad-independent (its change statistic depends only on nodal attributes and dyadic covariates of i and j, and not on other ties), the joint likelihood factors into independent per-dyad logistic terms, and the maximum pseudolikelihood estimate *is* the maximum likelihood estimate (van Duijn et al., 2009). Dyad-dependent terms, such as `mutual`, the geometrically weighted and shared-partner terms, the degree-distribution and triad terms, break the factorization, and $\kappa(\theta)$ cannot be avoided. `nwergm` then maximizes the likelihood by Monte Carlo: it simulates networks from a trial θ by a Metropolis–Hastings sampler and updates θ until the simulated statistics match the observed ones on average (Geyer and Thompson, 1992; Hummel et al., 2012). The log-likelihood at the estimate, needed for the information criteria, is evaluated by bridge sampling from the dyad-independent submodel.

4 The nwergm command

4.1 Syntax

Fitting a model

`nwergm` [*netname*]`, edges` [*terms*] [*options*]

Simulating networks from the last fit, or from given coefficients

`nwergm simulate` [`,` *simulate_options*]

`nwergm simulate` *nodes*`, edges` [*terms*] `theta(`*numlist*`)` [*simulate_options*]

Postestimation

`estat mcmcdiag` [, *mcmcdiag_options*]

`estat gof` [, *gof_options*]

`estat ic`

`predict,` {`matrix(`*name*`) | frame(`*name*`)`} [`pr | xb`]

A model is specified as a list of term options, the way a Stata regression command's option list specifies covariates. `edges` is required; *netname* defaults to the current network, so a model is fitted directly to whatever `nwset` or `nwwebuse` last produced. Node attributes are numeric variables of the current dataset (a string attribute is first converted with `encode`); dyadic covariates are networks in memory. Coefficients carry the names of `ergm`'s coefficients with the dot written as an underscore, such as `nodematch_seat`. An estimation result is an ordinary Stata estimation result: `e(b)`, `e(V)`, `estimates store`, `lrtest`, `test` and `predict` work as usual.

Network types. `nwergm` fits directed, undirected and two-mode (bipartite) networks. A directed two-mode network is refused, as are temporal networks. A network with tie values is fitted as the binary graph in which a tie is present when its value is not zero, as `ergm` does without its option `response=`; `nwergm` prints one note that the values are ignored (and no note when every value is 1). The terms for valued networks of `ergm` are not available (Section 10).

4.2 Term library

Table 1 lists the terms of `ergm` 4.12.0 by family and the number that `nwergm` implements. Of the 139 terms, 123 are implemented as in `ergm`, two in part, and 14 are not implemented (13 terms for valued networks and the operator `B`). Apart from these, the library is complete. Thirty-three of the terms apply to two-mode networks only. The definitions and the interpretation of the terms are those of `ergm` (Morris et al., 2008); Lusher et al. (2013) discuss when to prefer one over another. Figure 1 shows what six terms of the examples count.

How the terms are written. Each term is an option, and the common ones keep their R names: `mutual`, `nodematch()`, `nodecov()`, `degree()`, `gwesp()` and so on; Table 1 gives the spelling of every term. Two-mode terms move the mode digit to the end, because a Stata option name cannot have a digit followed by a letter: R's `b1degree` is `bdegree1()`. Less common structural terms are written in R's notation inside `structural()`, for example `structural(isolates cycle(k=3 4))`.

A geometrically weighted term such as `gwesp(.5)` has a fixed decay. `gwespfree(.5)` and the other `...free()` terms estimate the decay along with the other coefficients, starting from the number given. On a directed network the shared-partner terms take `type()`, with R's five definitions of a directed shared partner and R's default, `OTP`.

Table 1: The terms of `ergm` 4.12.0 by family, and how many `nwergm` implements.

Family	`ergm`	`nwergm`	Examples (`nwergm` syntax)
Density, reciprocity	5	5	`edges`, `mutual`, `hamming()`, `structural(density meandeg)`
Degrees, stars, concurrency	35	35	`degree()`, `odegree()`, `idegree()`, `kstar()`, `degrange()`, `concurrent`, `structural(isolates twopath degree1.5)`, two-mode `bdegree1()`, `bstar2()`
Shared partners	5	5	`esp()`, `dsp()`, `structural(nsp())`, `bdsp1()`; `type()` on directed networks
Triads, cycles, paths	18	18	`triangle`, `ctriple`, `transitiveties`, `cyclicalties`, `structural(cycle() ttriple threetrail simmelian triadcensus balance)`
Curved (geometrically weighted, alternating)	11	10 + 1 part	`gwesp()`, `gwdsp()`, `gwnsp()`, `gwdegree()`, `gwodegree()`, `gwidegree()`, two-mode `bgwdegree1()`, `bgwdsp1()`; estimated decay: `gwespfree()` and five more; `altkstar`
Node covariates	34	34	`nodecov()`, `nodefactor()`, `nodematch()`, `nodemix()`, `absdist()`, `sociality()`, `sender`, `receiver`, directed `nodeicov()`, `nodeofactor()`, two-mode `bcov1()`, `bfactor1()`
Dyadic covariates	3	3	`edgecov()`, `dyadcov()`, `attrcov()`
Operators	15	13 + 1 part	`operators(Sum() Prod() S() F() NodematchFilter() Symmetrize() Project() I() For() Label() Log() Exp() Offset() Curve())`
Valued networks only	13	0	—
All	139	123 + 2 part	125 of 139 (89.9%)

"part" means implemented in part: `altkstar` with an estimated decay (by pseudolikelihood only) and `Curve()` for linear maps (Section 10).

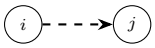
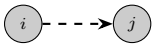
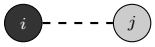
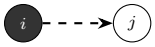
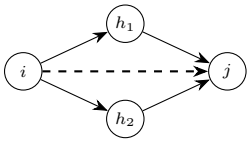
Diagram	Statistic $g(y)$	Description
	$\sum_{i,j} y_{ij}$	`edges`: the number of ties; the baseline tendency to form ties.
	$\sum_{i<j} y_{ij} y_{ji}$	`mutual`: i names j who already names i.
	$\sum_{i,j} y_{ij} [v_i = v_j]$	`nodematch(practicen)`: ties between lawyers in the same practice (here shaded alike).
	$\sum_{i<j} y_{ij} (w_i + w_j)$	`nodecov(wealth)`: the wealth of the two families of each marriage, added up.
	$\sum_{i,j} y_{ij} [v_i = 2]$	`nodeofactor(statusn)`: the ties sent by partners (level 2, dark).
	$\sum_{i,j} y_{ij}\, e^{\alpha}\{1-(1-e^{-\alpha})^{p_{ij}}\}$ with $p_{ij} = \sum_h y_{ih} y_{hj}$	`gwesp()`: the shared partners h of the tie $i \to j$, with declining weights, so a tie with many shared partners adds little beyond the first few.

Figure 1: What the terms count. The dashed tie is the one whose presence the term responds to; solid arrows are ties that already exist. The statistic is what the term adds up over the network ($y_{ij} = 1$ if i names j, v_i is a categorical attribute, w_i a numeric one, and α is the decay parameter of `gwesp()`). A tie that completes the configuration is more likely when the coefficient is positive. The terms are those of the examples in Section 7; `nodecov(wealth)` is drawn undirected, as the Florentine marriages are.

Operators. The option `operators()` applies terms to a derived network, as R's operator terms do. For example, `operators(Symmetrize(triangle, rule(strong)))` counts triangles in the network of reciprocated ties (Section 7.3), `S()` restricts a term to the subgraph of a set of nodes, and `Project()` evaluates it on the one-mode projection of a two-mode network. The manual lists all operators. Operators cannot be combined with constraints.

Constraints, offsets and observation. `constraints()` restricts the set of networks on which the model is defined, written as R's `constraints=`: for example `edges` (the number of ties is fixed), `degrees` or `bd()` (bounded degrees). A statistic that a constraint holds constant is dropped with a note, as in R. `obsconstraints()` describes how the network was observed, and `offset()` fixes a coefficient at a known value.

Missing data. As in R, a missing dyad is not treated as a non-tie. Missing dyads leave the pseudolikelihood; for a dyad-dependent model a second, constrained sample in which only the missing dyads are free enters the estimating equations (Handcock and Gile, 2010). `e(N)` is the number of informative dyads and `e(nmissing)` the number of missing dyads. A missing value in a node attribute or a dyadic covariate is an error, as in R.

4.3 Options

The options below are the ones most analyses need. Each control option has the name and default of R's `control.ergm()`; the manual lists the rest.

Model

`edges` includes the number of ties, the analogue of a constant. It is required.

terms are the model terms, written as options, such as `mutual`, `nodematch(`*varlist*`)` or `gwesp(#)`; Table 1 lists them.

`structural(`*termlist*`)` adds less common structural terms, written in R's notation.

`operators(`*oplist*`)` applies terms to a derived network.

`constraints(`*formula*`)` restricts the networks on which the model is defined, as R's `constraints=`, for example `constraints(edges)`.

`obsconstraints(`*formula*`)` describes how the network was observed, as R's `obs.constraints=`.

`offset(`*coefname* `# ...)` fixes coefficients at the values given instead of estimating them.

Estimation

`estimate(mle | mple | cd)` chooses the estimator. The default, `mle`, is the pseudolikelihood estimate for a dyad-independent model, where it is the maximum likelihood

estimate, and Monte Carlo maximum likelihood otherwise. `mple` forces the pseudolikelihood and `cd` gives contrastive divergence.

`mainmethod(mcmle | sa)` chooses the Monte Carlo algorithm: Monte Carlo maximum likelihood (the default) or stochastic approximation.

`seed(#)` sets the random-number seed, so that a Monte Carlo fit can be reproduced.

`init(`*numlist*`)` gives starting values, one per coefficient; a missing value leaves that coefficient to be estimated.

`mpletype(penalized)` gives the penalized pseudolikelihood estimate, which is finite when the ordinary one is not; `mplecovariance(godambe | bootstrap)` replaces its covariance.

Monte Carlo control

`mcmleeffectivesize(#)` is the effective sample size that the adaptive sampler aims for, 64 by default; `mcmleeffectivesize(0)` uses a sample of fixed size instead.

`mcmcsamplesize(#)`, `mcmcinterval(#)` **and** `mcmcburnin(#)` set the number of draws, the number of proposals between draws (1,024) and the burn-in (16 times the interval).

`mcmleiterations(#)` is the maximum number of iterations, 60 by default.

`nologlik` skips the evaluation of the log-likelihood after a Monte Carlo fit, which saves time when no information criteria are needed.

5 Estimation

Choosing the method. If every term is dyad-independent, maximum pseudolikelihood is the maximum likelihood estimate, and `nwergm` detects this automatically, reports the fit as an MPLE and runs no simulation. Any dyad-dependent term, a dyad-dependent constraint, a curved term or an observation constraint switches the default to Monte Carlo maximum likelihood, with the pseudolikelihood estimate as the starting value.

Pseudolikelihood. The pseudolikelihood is maximized by logistic regression of the ties on their change statistics. Options give the penalized estimate of Firth (1993), which is finite when the ordinary estimate is not, a check of whether the estimate exists, and Godambe or bootstrap covariances for dyad-dependent models, all as in R.

Monte Carlo maximum likelihood. The algorithm follows Hummel et al. (2012). Each iteration simulates networks at the current coefficients and moves the coefficients toward values at which the simulated statistics match the observed ones on average; the fit stops when a test says that they do, or after 60 iterations. The sampler proposes toggles as R's default does: most steps add or remove a tie (Morris et al., 2008), and the others toggle a dyad whose two nodes share a partner, which helps models with strong closure.

The controls have the names and defaults of R's `control.ergm()`, and the sample size adapts as in R: an iteration keeps sampling until the effective sample size of the estimating equations reaches a target (64 by default, `mcmleeffectivesize()`), and thins the chain as the sample grows. The number of draws and the interval of the final sample therefore come out of the fit; they are stored in `e()` and shown by `estat mcmcdiag`. `mcmleeffectivesize(0)` asks for samples of a fixed size instead. The log-likelihood is evaluated by bridge sampling, as in R, except after a curved Monte Carlo fit.

Contrastive divergence and stochastic approximation. `estimate(cd)` is contrastive divergence (Krivitsky, 2017): short chains started from the observed network, used to find a starting point or a rough estimate; the covariance reported is that of the short-chain sample. `mainmethod(sa)` is the stochastic approximation algorithm of Snijders (2002) in the form of R's `main.method="Stochastic-Approximation"`.

Reproducibility. With `seed()` the same call gives the same numbers in any session, on the same platform and plugin build. A different operating system, a different plugin build or a different value of a control option generally changes the chain and, within Monte Carlo error, the estimates.

6 Postestimation

estat mcmcdiag. After a Monte Carlo fit, `estat mcmcdiag` describes the final simulated sample: its size, interval and acceptance rate, and for each statistic the autocorrelation, the effective sample size and Geweke's test. Options add the Heidelberger–Welch test (`heidel`), the report of R's `mcmc.diagnostics()` (`report`) and trace and density plots (`plot`).

estat gof. `estat gof` follows R's `gof()`. It simulates networks from the fitted model and compares them with the observed network, on the model statistics and on features the model was not fitted to: degrees, shared partners, geodesic distances and the triad census. `plot` draws the box plots of R's `plot.gof()` (Hunter et al., 2008a).

Fit statistics and prediction. After every fit the log-likelihood, its Monte Carlo standard error, deviance, AIC and BIC are stored (BIC with N the number of informative dyads); `estat ic` displays them, `estimates stats` and `lrtest` compare models. `predict, matrix()` and `predict, frame()` return the conditional tie probabilities or log odds of each dyad given the rest of the observed network, as `predict.ergm`.

nwergm simulate. `nwergm simulate` draws networks from the last fit, as R's `simulate()` does, or, given a number of nodes, terms and coefficients, from a fully specified model. The draws can be kept as networks, as an edge list or as their statistics.

Options of the postestimation commands.

mcmcdiag_options `heidel` adds the Heidelberger–Welch test, `report` the report of R's `mcmc.diagnostics()` and `plot` the trace and density plots; `pvalue(#)` and `eps(#)` set the level and the precision of the Heidelberger–Welch test.

gof_options `gof(`*statistics*`)` chooses the comparisons from `model`, `degree`, `idegree`, `odegree`, `b1degree`, `b2degree`, `espartners`, `dspartners`, `distance` and `triadcensus`, with R's default for the type of network; `nsim(#)` sets the number of simulated networks (100); `seed(#)` sets the seed; `plot` draws the box plots.

simulate_options `nsim(#)` sets the number of networks (1); `output(network | stats | edgelist)` keeps them as networks, as their statistics or as an edge list; `generate(`*stub*`)` names them; `seed(#)` sets the seed; after a fit, `coef(`*matname*`)` simulates at other coefficients.

`pr` **and** `xb` give the conditional probability (the default) or the conditional log odds of each tie.

Stored results. `nwergm` stores the following in `e()`; the sampler results are those of a Monte Carlo fit.

Scalars

`e(N)`	number of informative dyads
`e(nodes)`	number of nodes
`e(ties)`	number of ties
`e(nmissing)`	number of missing dyads
`e(df_m)`	number of coefficients
`e(ll)`	log-likelihood
`e(ll_se)`	Monte Carlo standard error of `e(ll)`
`e(aic)`	AIC
`e(bic)`	BIC
`e(curved)`	1 if the model is curved
`e(converged)`	1 if the Monte Carlo fit converged
`e(mcmle_iterations)`	number of Monte Carlo iterations
`e(mcmc_samplesize)`	number of draws in the final sample
`e(mcmc_interval_final)`	interval of the final sample
`e(mcmc_acceptrate)`	acceptance rate of the sampler

Macros

`e(cmd)`	`nwergm`
`e(depvar)`	name of the network
`e(directed)`	`true` or `false`
`e(estimate)`	estimator
`e(method)`	`mple` or `mcmle`
`e(version)`	version of `nwergm`

Matrices

`e(b)`	coefficient vector
`e(V)`	variance–covariance matrix
`e(mcmcsample)`	statistics of the final sample

`help nwergm` lists all stored results. Report `e(version)` with an analysis.

7 Remarks and examples

The main example follows one question through the whole workflow: fitting a model, reading its coefficients, checking the simulation behind them and checking the fit.

Shorter examples then show two-mode networks, curved terms, operators and simulation. Every Monte Carlo fit has a `seed()`, so the logs can be reproduced.

7.1 Advice among lawyers

Lazega and van Duijn (1997) studied a corporate law firm in New England with 71 lawyers: 36 partners and 35 associates, in three offices and two practices (litigation and corporate law). Each lawyer was asked whom they turn to for professional advice. An arc from i to j means that i seeks advice from j. The data come with `nwcommands`: `nwwebuse` loads the advice network and the lawyers' attributes, and `nwsummarize` describes the network. The factor terms below need numeric attributes, so `encode` turns office, practice and status into numeric variables, and `nwdyads` counts the pairs of lawyers by their ties:

```
. nwwebuse lazega, nwclear
. nwsummarize lazega_adv
```

```
  Network name:  lazega_adv
  Network id:  1
  Directed: true
  Valued: false
  Two-mode: false
  Nodes: 71
  Selfloop: false
  Arcs: 892
  Minimum value:  0
  Maximum value:  1
  Density:  .179
  Temporal: false
```

```
. foreach v in office practice status {
  2.     encode `v´, generate(`v´n)
  3. }
. nwdyads lazega_adv
```

```
   Dyad census:  lazega_adv
   Mutual |       Asym |       Null
----------+------------+------------
      192 |        508 |       1785

   Reciprocity: .274
```

Of the $71 \times 70 = 4{,}970$ possible arcs, 892 are present, a density of 0.18. Of the 700 pairs with at least one arc, 192 are mutual (the reciprocity of 0.27). Figure 2 shows the network: the Boston lawyers form one large cluster, and those in Hartford and Providence lie to the side.

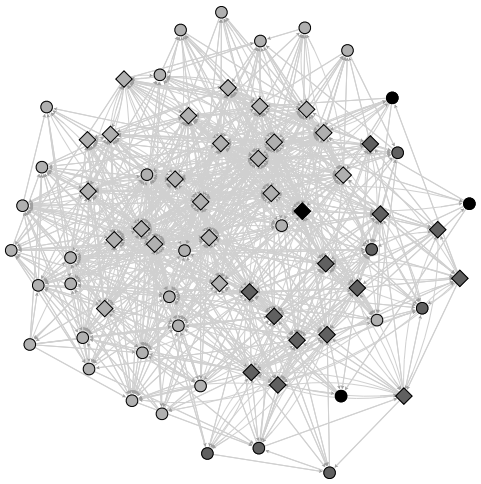


Figure 2: Advice network among 71 lawyers (`nwplot lazega_adv`, do-file `plot_networks.do`). Shading shows the office (light grey Boston, mid grey Hartford, black Providence), diamonds partners and circles associates. Arrowheads are too small to read at this density.

A first model. Do lawyers seek advice within their own office and practice? And do partners seek and give more advice than associates? The first model has four terms besides `edges`: `nodematch()` counts the arcs between lawyers in the same office or practice, `nodeofactor(statusn)` counts the arcs sent by partners, and `nodeifactor(statusn)` counts the arcs received by partners:

```
. nwergm lazega_adv, edges nodematch(officen practicen) ///
>     nodeofactor(statusn) nodeifactor(statusn)

Exponential-family random graph model

Network:               =  lazega_adv
Nodes:                 =  71
Ties:                  =  892
Directed:              =  Yes
Estimation:            =  MPLE
```

lazega_adv	Coefficient	Std. err.	z	P>|z|	[95% conf.	interval]
edges	-4.095448	.1309373	-31.28	0.000	-4.352081	-3.838816
nodematch_o~n	1.422373	.0887433	16.03	0.000	1.248439	1.596306
nodematch_p~n	1.037945	.0837376	12.40	0.000	.8738223	1.202068
nodeofactor~2	1.125832	.0842818	13.36	0.000	.9606422	1.291021
nodeifactor~2	.7831546	.0819723	9.55	0.000	.6224919	.9438172

```
Maximum pseudolikelihood estimate; this is the maximum likelihood estimate
```

```
only if the model is dyad-independent.
            Null Deviance:    6889.9  on 4970 degrees of freedom
        Residual Deviance:    4009.0  on 4965 degrees of freedom
Log-likelihood: -2004.4927 (df = 5)   AIC:    4019.0   BIC:    4051.5
. estimates store m1
```

Each term in this model depends only on the attributes of the two lawyers in a pair, not on other ties. Such a model is a logistic regression with one observation per ordered pair of lawyers, and the pseudolikelihood estimate is the maximum likelihood estimate. The coefficients are read as in a logit model.

The `edges` coefficient plays the role of the constant. It is the log odds of an arc between two associates in different offices and practices: -4.10, a probability of 0.016. The other coefficients add to these log odds. An arc between two associates in the same office and practice has log odds $-4.10 + 1.42 + 1.04 = -1.64$, a probability of 0.16. If the arc goes from a partner to a partner, the two status terms add $1.13 + 0.78$, and the probability is 0.57. In odds ratios, sharing an office multiplies the odds of an advice arc by $\exp(1.42) = 4.1$ and sharing a practice by 2.8. Partners send arcs with 3.1 times the odds of associates and receive them with 2.2 times the odds.

Adding network dependence. The first model assumes that the arcs are independent once the attributes are taken into account. Advice is unlikely to work that way. A lawyer who is asked for advice may ask back, which is reciprocity. And a lawyer who asks a colleague, who in turn asks a third, may come to ask the third directly, which is closure. The second model adds `mutual` (not shown), and the third also adds `gwesp(.5)`:

```
. nwergm lazega_adv, edges mutual nodematch(officen practicen) ///
>     nodeofactor(statusn) nodeifactor(statusn) gwesp(.5) seed(12345)
Exponential-family random graph model
Network:               =  lazega_adv
Nodes:                 =  71
Ties:                  =  892
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  541
```

lazega_adv	Coefficient	Std. err.	z	P>|z|	[95% conf.	interval]
edges	-4.662469	.1821854	-25.59	0.000	-5.019546	-4.305392
mutual	1.031625	.1244439	8.29	0.000	.7877192	1.27553
nodematch_o~n	1.016895	.0839039	12.12	0.000	.852446	1.181343
nodematch_p~n	.7571778	.0784148	9.66	0.000	.6034877	.910868
nodeofactor~2	.8242996	.0907388	9.08	0.000	.6464549	1.002144
nodeifactor~2	.4019506	.0892556	4.50	0.000	.2270129	.5768884
gwesp_.5	.5597618	.1091626	5.13	0.000	.345807	.7737166

```
MCMLE converged after 5 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...
            Null Deviance:    6889.9  on 4970 degrees of freedom
        Residual Deviance:    3908.6  on 4963 degrees of freedom
Log-likelihood: -1954.3217 (df = 7)   AIC:    3922.6   BIC:    3968.2
Monte Carlo standard error of the log-likelihood:  .10671
```

```
. estimates store m3
```

With these terms, an arc depends on the other arcs. The model is now fitted by Monte Carlo maximum likelihood, and each coefficient describes the log odds of an arc *given the rest of the network*.

The `mutual` coefficient says that an arc from i to j has $\exp(1.03) = 2.8$ times the odds if j already seeks advice from i. The `gwesp` term counts shared partners: for an arc from i to j, the third lawyers k with $i \to k \to j$. With the decay of 0.5, the first shared partner of an arc counts 1 in the statistic, the second adds 0.39 and the third 0.15. The positive coefficient (0.56, $z = 5.1$) means that an arc whose two lawyers already share a partner is more likely than an arc that would close no triad, by a factor of about $\exp(0.56) = 1.8$ for the first shared partner, and less for each further one.

Office homophily falls from 1.42 to 1.02, and the status effects fall too. Part of what the first model attributed to the attributes is reciprocity and closure: lawyers in the same office trade advice and form triads, and the first model counted these extra arcs as homophily. The office effect remains large ($\exp(1.02) = 2.8$).

Checking the simulation. A Monte Carlo estimate is only as good as the simulated networks behind it. `estat mcmcdiag` describes the final sample:

```
. estat mcmcdiag
MCMC diagnostics, final sample                Sample size      =        541
                                              Burn-in          =     16,384
                                              Interval         =      4,096
                                              Acceptance rate  =     0.4998
```

Statistic	Mean	SD	Autocorr	ESS	Geweke z
edges	890.97	27.62	0.218	348.4	-1.11
mutual	190.45	12.11	0.219	347.1	-1.09
nodematch_officen	671.10	23.57	0.259	319.5	-0.66
nodematch_practicen	609.75	21.37	0.301	292.2	0.39
nodeofactor_statusn_2	616.28	20.71	0.236	335.4	-2.25*
nodeifactor_statusn_2	561.06	19.25	0.231	338.7	-0.76
gwesp_.5	1282.16	55.17	0.207	356.3	-1.09

```
The adaptive sampler raised the interval from 1,024 to 4,096.
* |z| > 1.96: the mean of the first 10% of the sample differs from the mean of
  the last 50% at the 5% level (Geweke 1992).

The convergence test of MCMLE was satisfied after 5 iterations. This is
necessary but not sufficient for convergence; option plot draws the trace
plots.
If the ESS is low, raise mcmleeffectivesize(); after mcmleeffectivesize(0),
raise mcmcsamplesize() or mcmcinterval().
```

The sample has 541 networks. Successive networks are similar, so the sample carries less information than 541 independent draws: the effective sample size, about 290 to 360, is what the sample is worth. The Geweke test compares the start and the end of the sample. One of the seven statistics is flagged at the 5% level ($z = -2.3$), which is not unusual among seven tests; the Heidelberger–Welch test (option `heidel`, not shown) passes for all seven. Figure 3 shows the traces, which move around a stable level without drifting or sticking.

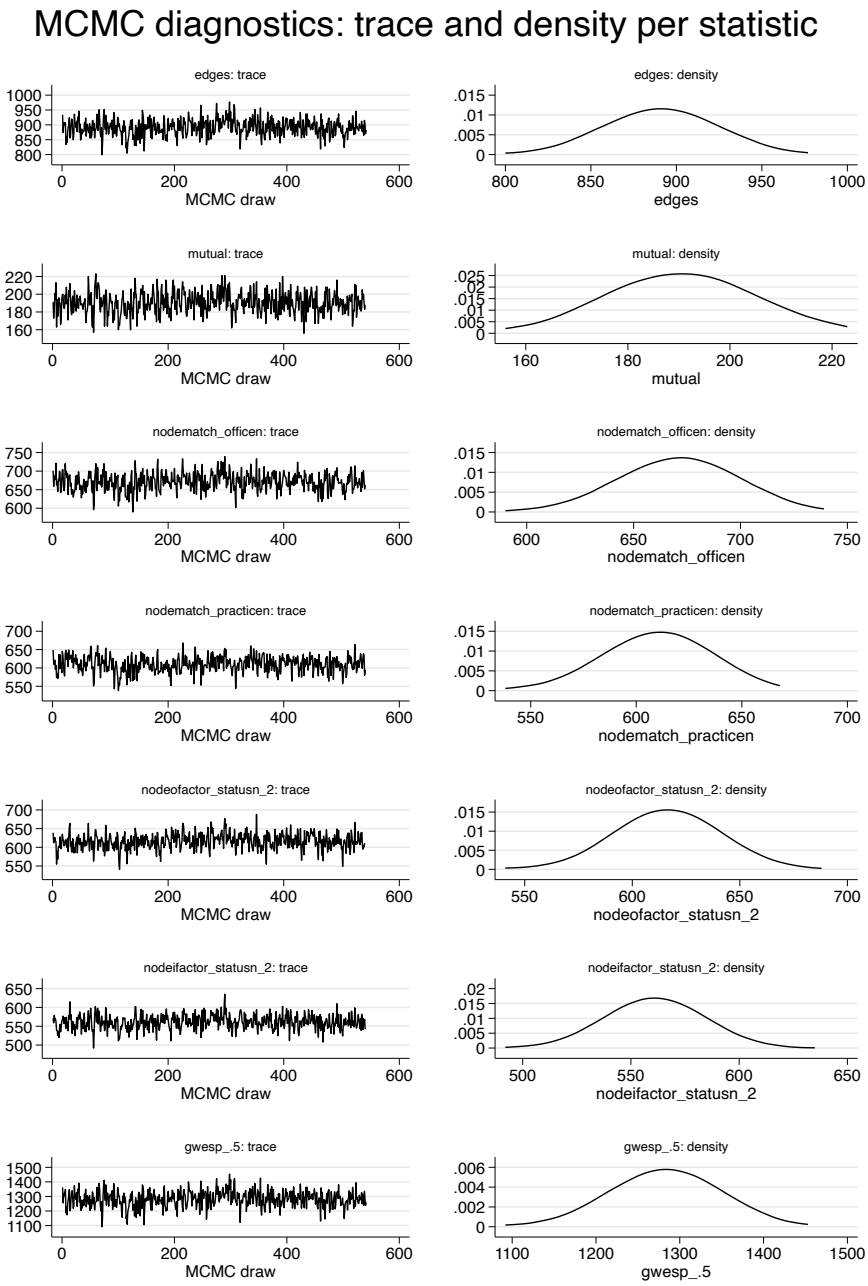


Figure 3: `estat mcmcdiag, plot` after the third advice model: trace (left) and density (right) of each model statistic in the final sample.

Comparing the models. Information criteria and a likelihood-ratio test compare the three models:

```
. estimates stats m1 m2 m3
Akaike´s information criterion and Bayesian information criterion
```

Model	N	ll(null)	ll(model)	df	AIC	BIC
m1	4,970	-3444.941	-2004.493	5	4018.985	4051.541
m2	4,970	-3444.941	-1971.793	6	3955.585	3994.652
m3	4,970	-3444.941	-1954.322	7	3922.643	3968.222

```
Note: BIC uses N = number of observations. See [R] IC note.
. lrtest m2 m3
Likelihood-ratio test
Assumption: m2 nested within m3
 LR chi2(1) =  34.94
Prob > chi2 = 0.0000
```

Each model improves on the one before, and closure improves on reciprocity alone ($\chi^2 = 34.9$ on 1 degree of freedom).

Checking the fit. The model was fitted to its own statistics, so it reproduces them on average. The real test is whether networks simulated from it also look like the observed network in other respects. `estat gof` simulates 100 networks and compares them with the observed one, here on the model statistics and on the geodesic distances, the number of steps from one lawyer to another:

```
. estat gof, gof(model distance) seed(12345)
Goodness of fit: model statistics (100 simulated networks)
```

Statistic	Observed	Min	Mean	Max	MC p-value
edges	892	819	893.20	993	0.96
mutual	192	161	192.55	233	0.98
nodematch_officen	674	608	679.34	752	0.80
nodematch_practicen	610	561	611.17	711	0.98
nodeofactor_statusn_2	616	570	615.94	667	0.92
nodeifactor_statusn_2	562	516	561.23	642	0.94
gwesp_.5	1284.86	1134.59	1288.29	1496.89	0.94

```
Goodness of fit: minimum geodesic distance (100 simulated networks)
```

Distance	Observed	Min	Mean	Max	MC p-value
1	892	819	893.20	993	0.96
2	3074	3104	3282.93	3461	0.00
3	991	527	729.97	963	0.00
4	13	0	16.97	70	1.00
5	0	0	0.14	3	1.00
Inf	0	0	46.79	209	1.00

The model statistics are reproduced, and most distance counts lie within the simulated range. Two do not. The simulated networks have too many pairs two steps apart (3,283

against 3,074 observed) and too few three steps apart (730 against 991). The real firm is less tightly knit than the model implies, and a next model might look for what keeps some lawyers apart, such as seniority. Figure 4 shows the same comparison for the degrees and the distances: the observed degrees lie mostly inside the simulated boxes.

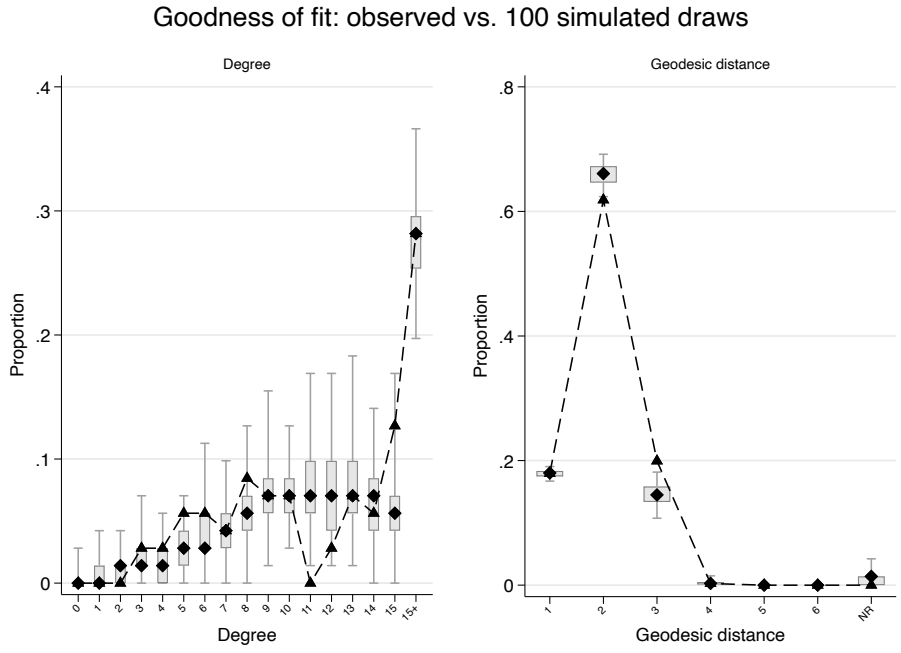


Figure 4: `estat gof, plot` after the third advice model: degree (left) and geodesic-distance (right) distributions, the observed network (dashed line) against 100 simulated networks (boxes and whiskers).

7.2 A two-mode network: the Southern women

The 18 women and 14 social events of the classic data set of Davis et al. (2022) form a two-mode network of 89 attendances (analyzed in Breiger, 1974). With `title` (Miss or Mrs) as the only attribute of the women, the terms of two-mode networks count the nodes of one mode: `bfactor1()` is R's `b1factor`, and `bgwdegree2()` and `bgwdsp1()` are the geometrically weighted degree of the events and the geometrically weighted dyadwise shared partners of the women. The data are loaded in the same way:

```
. nwwebuse davis, nwclear
. encode title, generate(titlen)
```

Three models with the same attribute and different structural terms were fitted; the listing shows the shared-partner model and the information criteria of all three:

```
. nwergm davis, edges bfactor1(titlen) bgwdsp1(.5) seed(1)
Exponential-family random graph model
Network:               =  davis
Nodes:                 =  32
Ties:                  =  89
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  486
```

davis	Coefficient	Std. err.	z	P>|z|	[95% conf.	interval]
edges	-1.176527	.2987292	-3.94	0.000	-1.762026	-.5910288
gwb1dsp_fix~5	.3024289	.1292092	2.34	0.019	.0491835	.5556743
b1factor_ti~2	-.0664987	.3370617	-0.20	0.844	-.7271276	.5941301

```
MCMLE converged after 2 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...
            Null Deviance:     349.3  on 252 degrees of freedom
        Residual Deviance:     321.9  on 249 degrees of freedom
Log-likelihood:  -160.9260 (df = 3)   AIC:     327.9   BIC:     338.4
Monte Carlo standard error of the log-likelihood:  .04524
. estimates store d3
. estimates stats d1 d2 d3
Akaike´s information criterion and Bayesian information criterion
```

Model	N	ll(null)	ll(model)	df	AIC	BIC
d1	252	-174.6731	-163.6325	2	331.2651	338.324
d2	252	-174.6731	-161.9122	3	329.8244	340.4127
d3	252	-174.6731	-160.926	3	327.8519	338.4402

```
Note: BIC uses N = number of observations. See [R] IC note.
```

Whether a woman is a Miss or a Mrs has no effect on how many events she attends. The shared-partner term counts, for each pair of women, the events both attended. Its positive coefficient (0.30, $z = 2.3$) means that attendance is clustered: women tend to turn up at the same events as each other, more than the overall rate of attendance implies, and the degrees of the events are more unevenly distributed than a model with only a density term implies (the coefficient of `bgwdegree2` is -2.2, $z = -2.6$ in the second model, not shown); the information criteria slightly prefer the shared-partner model. Its goodness of fit adds the degree distributions of both modes:

```
. estat gof, seed(1)
Goodness of fit: model statistics (100 simulated networks)
```

Statistic	Observed	Min	Mean	Max	MC p-value
edges	89	78	89.74	106	1.00
gwb1dsp_fixed_.5	185.26	147.02	185.62	220.81	0.84
b1factor_titlen_2	34	27	34.68	45	1.00

```
Goodness of fit: minimum geodesic distance (100 simulated networks)
```

Distance	Observed	Min	Mean	Max	MC p-value

1	89	78	89.74	106	1.00
2	205	185	219.71	240	0.14
3	163	137	160.79	174	0.82
4	39	4	23.60	44	0.12
5	0	0	0.61	5	1.00
Inf	0	0	1.55	31	1.00

Goodness of fit: bipartition 1 degree (100 simulated networks)

Degree	Observed	Min	Mean	Max	MC p-value
0	4	4	4.15	5	1.00
1	0	0	0.48	4	1.00
2	2	0	1.20	5	0.62
3	3	0	1.97	6	0.58
4	2	0	2.37	7	1.00
5	2	0	2.32	6	1.00
6	2	0	2.09	5	1.00
7	1	0	1.52	4	1.00
8	0	0	0.90	3	0.78
9	0	0	0.62	3	1.00
10	0	0	0.24	2	1.00
11	2	0	0.11	1	0.00
12	0	0	0.02	1	1.00
13	0	0	0.01	1	1.00

Goodness of fit: bipartition 2 degree (100 simulated networks)

Degree	Observed	Min	Mean	Max	MC p-value
0	0	0	0.01	1	1.00
1	0	0	0.03	1	1.00
2	2	0	0.36	2	0.08
3	1	0	1.60	6	1.00
4	4	0	3.45	7	0.92
5	1	0	3.81	8	0.12
6	1	0	2.63	6	0.50
7	2	0	1.45	4	0.96
8	3	0	0.48	2	0.00
9	0	0	0.16	2	1.00
10	0	0	0.02	1	1.00

Goodness of fit: dyadwise shared partner (100 simulated networks)

DSP	Observed	Min	Mean	Max	MC p-value
0	91	68	86.79	108	0.56
1	40	24	54.88	90	0.16
2	50	38	54.82	72	0.50
3	32	12	31.67	53	1.00
4	17	2	11.69	33	0.32
5	7	0	3.24	11	0.22
6	5	0	0.72	4	0.00
7	1	0	0.14	2	0.24
8	1	0	0.03	1	0.06
9	0	0	0.01	1	1.00
11	0	0	0.01	1	1.00

The statistics of the model and most counts of the degree and shared-partner distributions are reproduced. Three counts at the upper end are not: the two women who attended 11 events (simulated mean 0.11), the three events with 8 attendees (0.48) and the five dyads with six shared partners (0.72) all have a Monte Carlo p-value of 0.00, so the model underproduces the extremes. With 18 women and 14 events the example illustrates the syntax and the output; it is too small for a substantive finding.

7.3 Curved terms and operators

A geometrically weighted term with an estimated decay is a curved term. On the friendship network of the students of a school (Faux Mesa High, 205 students, 203 ties, `mesa`), the closure term `gwespfree(.25)` estimates a weight and a decay; the number in parentheses is the starting value. This is a Monte Carlo fit that takes about 2 seconds on the machine used:

```
. nwwebuse mesa, nwclear
. encode race, generate(racen)
. nwergm mesa, edges nodematch(grade racen) gwespfree(.25) seed(1)
Exponential-family random graph model
Network:               =  mesa
Nodes:                 =  205
Ties:                  =  203
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  1203
```

mesa	Coefficient	Std. err.	z	P>|z|	[95% conf.	interval]
edges	-6.367347	.1666168	-38.22	0.000	-6.69391	-6.040784
nodematch_g~e	1.964598	.1758003	11.18	0.000	1.620036	2.30916
nodematch_r~n	.276849	.1214278	2.28	0.023	.0388548	.5148431
gwesp_weight	1.35144	.1369479	9.87	0.000	1.083027	1.619853
gwesp_decay	.3820264	.1056824	3.61	0.000	.1748926	.5891602

```
MCMLE converged after 12 iteration(s).
(The decay of the curved term is estimated; see help nwergm, gw_effects.)
```

Students in the same grade are friends with $\exp(1.96) = 7.1$ times the odds of students in different grades, and students of the same race with 1.3 times the odds. The closure term now has two parameters: a weight of 1.35 for the first shared partner and a decay of 0.38 that says how quickly further shared partners count for less. With a fixed decay, as in the advice model, the analyst chooses it; here the data do. The likelihood and the information criteria are not available after a curved fit by Monte Carlo maximum likelihood.

An operator evaluates a term on a derived network. The advice network is a directed network; on the friendship network of the lawyers, whether friendship triads close among *reciprocated* friendships alone is asked by applying `triangle` to the undirected network in which two lawyers are tied if the friendship is mutual:

```
. nwwebuse lazega, nwclear
. foreach v in office practice status {
```

```
  2.     encode `v´, generate(`v´n)
  3. }
. nwergm lazega_fr, edges mutual nodematch(practicen) ///
>     operators(Symmetrize(triangle, rule(strong))) seed(1)
Exponential-family random graph model
Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  766
```

lazega_fr	Coefficient	Std. err.	z	P>|z|	[95% conf.	interval]
edges	-2.736795	.0810857	-33.75	0.000	-2.89572	-2.57787
mutual	.9130825	.1676719	5.45	0.000	.5844516	1.241713
nodematch_p~n	.8967159	.0964408	9.30	0.000	.7076954	1.085736
Symmetrize(~e	.6922598	.4560459	1.52	0.129	-.2015737	1.586093

```
MCMLE converged after 5 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...
             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3401.6  on 4966 degrees of freedom
Log-likelihood: -1700.7989 (df = 4)   AIC:    3409.6   BIC:    3435.6
Monte Carlo standard error of the log-likelihood:  .10562
```

Closure among mutual friendships is positive but not significant at the 5% level (0.69, $z = 1.5$) on top of reciprocity and same-practice homophily.

7.4 Simulating from given coefficients

`nwergm simulate` with a number of nodes draws networks from a fully specified model; the coefficients are given in the order of the terms:

```
. nwergm simulate 16, edges gwesp(.5) theta(-2 .5) seed(42) nsim(3)
. nwsummarize ergmsim_1
```

```
  Network name:  ergmsim_1
  Network id:  4
  Directed: false
  Valued: false
  Two-mode: false
  Nodes: 16
  Selfloop: false
  Edges: 27
  Minimum value:  0
  Maximum value:  1
  Density:  .225
  Temporal: false
```

8 Implementation

Native plugin. The sampler and the change statistics of the terms are written in C and run in a Stata plugin; the estimating steps, the covariance, the diagnostics and

the output are written in Mata and Stata. The plugin is built for macOS, Linux and Windows and installed with the package. The test suite passes on macOS and Windows; the Linux plugin has not yet been tested on a Linux machine. The C source is part of the package, and `nwinstall, buildplugin(nwergm)` compiles it on a computer where the shipped plugin does not work.

A slower Mata version of the original terms is kept for computers without the plugin. On the Florentine marriage network (Padgett and Ansell, 1993) with `gwesp(.5)` it takes 16 seconds instead of 0.2, and `nwergm` prints a warning when it is used. The terms in `structural()`, the constraints and the operators need the plugin.

Mata library. The commands share a Mata library that is compiled against the core library of `nwcommands`. At the start of a command the package checks that the two match and, if they do not, prints how to reinstall. Both libraries are built with Stata 17 and work in Stata 17, 18 and 19.

9 Comparison with R

9.1 How the terms were validated

`nwergm` was developed against `ergm` 4.12.0 (R 4.6.0). Each term was checked in three steps. Its observed statistic had to equal R's on at least two networks of each applicable type, to 10^{-6}. Its pseudolikelihood coefficients and standard errors had to equal R's to 10^{-4}. Its Monte Carlo estimates were fitted with several seeds in each program, and the means were compared with the seed-to-seed spread and the standard errors; Section 9.2 reports the results, including three coefficients whose difference exceeds the Monte Carlo error. For the two-mode terms the whole matrix of change statistics was compared.

One term differs on purpose. The change statistic of `opentriad` in `ergm` 4.12.0 is wrong; `nwergm` uses the correct one, so its observed statistic equals R's but its estimates do not. The checks are part of the test suite of the package, which runs before every release.

9.2 Estimates on the same data

Table 2 fits the models of Section 7 and two small networks (a five-node network and the Florentine marriages) in `ergm` 4.12.0 and `nwergm` on the same networks, with the default settings of each program. For the Monte Carlo fits, each program was run with several seeds; the table gives the mean over seeds and the seed-to-seed standard deviation of each program, and the difference between the means in units of the standard error that `ergm` reports.

For the models fitted by pseudolikelihood the two programs give the same numbers, to 3×10^{-8} in the coefficients and 10^{-5} in the standard errors.

For the Monte Carlo fits, the means of the two programs differ by at most 0.09 standard errors in every coefficient, and their standard errors agree within 4%. The seed-to-seed spread is small in both programs: in the final advice model it is 0.002 to 0.010, against standard errors of 0.07 to 0.18.

A stricter test compares the difference between the two means with their Monte Carlo

Table 2: Coefficients of `ergm` 4.12.0 and `nwergm` 1.5.4 on the same networks, default settings. Mean over seeds (standard deviation over seeds in parentheses); Δ is the difference of the means in units of the standard error that `ergm` reports.

	`ergm`		`nwergm`		
Term	mean	(sd)	mean	(sd)	Δ
5-node directed network, MCMLE (10 seeds)					
edges	-0.954	(0.050)	-0.953	(0.042)	0.00
mutual	0.285	(0.093)	0.306	(0.095)	0.01
Florentine marriages, MPLE					
edges	-2.595		-2.595		0.00
nodecov(wealth)	0.011		0.011		0.00
Florentine marriages, MCMLE (10 seeds)					
edges	-2.586	(0.044)	-2.582	(0.028)	0.01
nodecov(wealth)	0.011	(0.000)	0.011	(0.000)	-0.01
gwesp(.5)	-0.011	(0.020)	-0.016	(0.022)	-0.02
Lazega advice, MPLE					
edges	-4.095		-4.095		0.00
nodematch(office)	1.422		1.422		0.00
nodematch(practice)	1.038		1.038		0.00
nodeofactor(status=2)	1.126		1.126		0.00
nodeifactor(status=2)	0.783		0.783		0.00
Lazega advice, MCMLE (5 seeds)					
edges	-4.648	(0.007)	-4.663	(0.008)	-0.08
mutual	1.034	(0.010)	1.025	(0.004)	-0.07
nodematch(office)	1.006	(0.005)	1.006	(0.005)	0.00
nodematch(practice)	0.759	(0.006)	0.756	(0.004)	-0.04
nodeofactor(status=2)	0.821	(0.004)	0.818	(0.005)	-0.04
nodeifactor(status=2)	0.397	(0.005)	0.398	(0.004)	0.01
gwesp(.5)	0.557	(0.003)	0.566	(0.002)	0.09
Lazega friendship, operator (see text)					
edges	-2.736	(0.007)	-2.739	(0.002)	-0.04
mutual	0.921	(0.019)	0.925	(0.008)	0.03
nodematch(practice)	0.895	(0.002)	0.897	(0.004)	0.01
Symmetrize(strong)∼triangle	0.671	(0.036)	0.697	(0.031)	0.05
Southern women, two-mode MCMLE (10 seeds)					
edges	-0.316	(0.013)	-0.327	(0.017)	-0.06
b1factor(title=2)	-0.045	(0.022)	-0.044	(0.021)	0.00
gwb2degree(1)	-2.360	(0.053)	-2.295	(0.054)	0.07
Faux Mesa High, MCMLE (5 seeds)					
edges	-6.412	(0.004)	-6.413	(0.007)	-0.01
nodematch(grade)	1.991	(0.005)	1.988	(0.007)	-0.02
nodematch(race)	0.285	(0.003)	0.288	(0.006)	0.02
gwesp(.25)	1.452	(0.007)	1.450	(0.003)	-0.02
Faux Mesa High, curved gwesp (see text)					
edges	-6.369		-6.368	(0.001)	0.00
nodematch(grade)	1.973		1.971	(0.006)	-0.01
nodematch(race)	0.279		0.274	(0.003)	-0.05
gwesp weight	1.343		1.346	(0.005)	0.02
gwesp decay	0.389		0.388	(0.005)	-0.02

error, the seed-to-seed standard deviation divided by the square root of the number of seeds. Of the 23 coefficients fitted with several seeds in both programs, 20 differ by less than two such errors. Three differ by more: `gwesp` and `edges` in the final advice model (5.4 and 3.1 errors, or 0.010 and 0.014 in the coefficient) and `gwb2degree` in the Southern women model (2.7 errors, or 0.065). So small differences between the implementations remain in some models. At under a tenth of a standard error, they should not change what one concludes from these models.

Two rows rest on fewer `ergm` fits. On the Lazega friendship model, `ergm` stopped for one of five seeds because its sampler did not mix. On the curved model for Faux Mesa High, it stopped for two of three seeds because a dyad had more than 30 shared partners, the default `cutoff` of its curved term. The single `ergm` fit and the mean of the three `nwergm` fits differ by 0.05 standard errors or less; with one fit, the spread of `ergm` on this model is unknown. All `nwergm` fits converged.

9.3 Speed

Table 3 gives the median time of seven fits of each program on the same networks, each with its default settings. The machine was an Apple M5 Pro (18 cores, macOS 26.5.1) with R 4.6.0, `ergm` 4.12.0 and Stata/BE 18, with no other Stata or R job running. The two programs took turns, one fit at a time, so that any drift of the machine affected both. Each fit ran in its own process after an untimed warm-up, and only the estimation call was timed. Four networks are stored in the replication files, two are generated with fixed seeds, and two are the Lazega and Faux Mesa High models of Section 7.

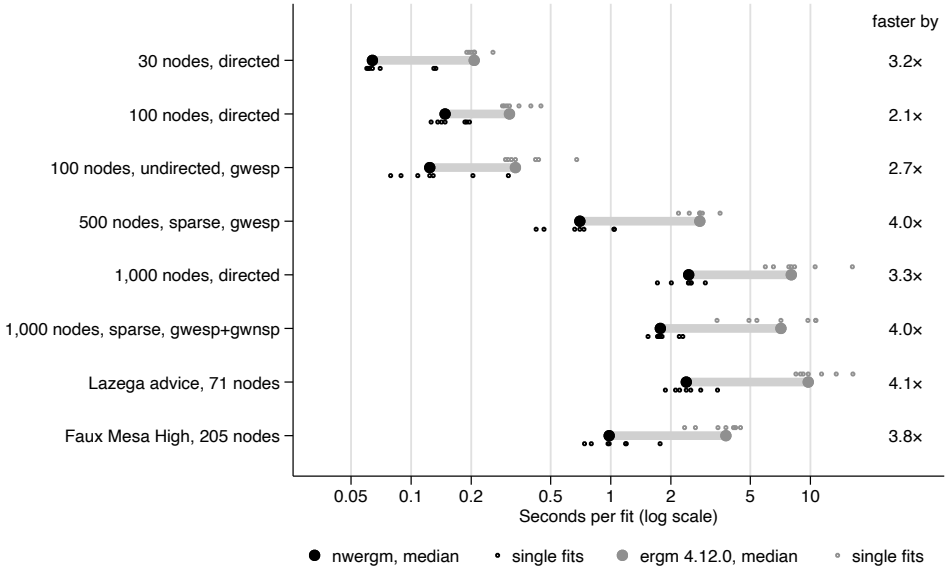


Figure 5: Time of seven fits of each benchmark model (seeds 1 to 7) with `nwergm` (black) and `ergm` 4.12.0 (grey), default settings, log scale. Small circles are single fits, large circles the medians; the number at the right is the ratio of the medians. The numbers are in Table 3.

Table 3: Median time of seven fits (seeds 1 to 7, fitted in turn on the same machine) of `ergm` 4.12.0 and of `nwergm` on identical networks, default settings; ratio of the medians, `ergm` divided by `nwergm`; Monte Carlo iterations in the seven fits of each program (minimum to maximum).

Benchmark	`ergm` (s)	`nwergm` (s)	ratio	`ergm` it.	`nwergm` it.
30 nodes, directed: edges, mutual	0.21	0.06	3.2	1	1
100 nodes, directed: + nodematch	0.31	0.15	2.1	1–3	1–4
100 nodes, undirected: edges, gwesp	0.33	0.12	2.7	1	1
500 nodes, sparse, undir.: + nodematch, gwesp	2.8	0.70	4.0	3–5	2–6
1000 nodes, directed: edges, mutual, nodematch	8.0	2.5	3.3	2–3	1–3
1000 nodes, sparse, undir.: edges, gwesp, gwnsp	7.1	1.8	4.0	2–6	2–4
Lazega advice, 71 nodes: seven terms	9.8	2.4	4.1	6–8	6–9
Faux Mesa High, 205 nodes: gwesp(.25)	3.8	0.98	3.8	5–11	6–10

`nwergm` was faster on all eight models (Figure 5), by factors of 2.1 to 4.1. The two networks of 1,000 nodes took 2.5 and 1.8 seconds against 8.0 and 7.1; the small networks took 0.06 to 0.15 seconds against 0.21 to 0.33, much of it the fixed cost of a call. Times vary with the seed in both programs, because the number of iterations does, so a single fit can come out the other way; the ratios are those of the medians.

The measurements suggest that most of the gain comes from a cheaper proposal rather than from less sampling. Both programs run the same algorithm with the same defaults (Section 5), and they need similar numbers of iterations. I counted the Metropolis–Hastings proposals of three seeds per model with a counter added to a copy of each program: `nwergm` made 0.7 to 1.5 times as many as `ergm`, following the number of iterations. Per proposal, `nwergm` is faster: with a fixed workload, one costs 0.08 to 0.25 microseconds in `nwergm` and 0.20 to 1.04 in `ergm`.

Models with one coefficient per node. `sender` and `receiver`, and factor terms with many levels, give models with hundreds or thousands of coefficients. Without a dyad-dependent term, such a model is fitted exactly by the pseudolikelihood with a sparse design: `edges sender receiver` on 400 nodes (799 coefficients) took 1.0 second, against 573 seconds in `ergm`, and on 1,000 nodes about 8 seconds. With a dyad-dependent term, both programs draw at least 32 networks per coefficient, and such fits take much longer.

After the fit. On large networks the steps after a fit can take longer than the fit. On 1,000 nodes with `edges` and `gwesp`, the fit took 1 second, `estat gof` 39 seconds and simulating 100 networks 28 seconds.

Scope. The timings come from one machine with Stata 18 on macOS and hold for these eight models. The speed relative to `ergm` depends on the model and the data, and a model that runs in the Mata sampler is much slower than `ergm`.

10 What is not implemented

Of the 139 terms of `ergm` 4.12.0, 14 are not implemented: the 13 terms for valued networks and the operator `B.` `nwergm` models whether a tie exists; a valued network is fitted as a binary one (Section 4). Two terms are implemented in part: `Curve()` supports only linear maps, and `altkstar` with an estimated decay can be fitted only by pseudolikelihood (and `ergm` itself cannot fit it by Monte Carlo on the test networks).

A few combinations are also missing: some arguments of the operators, four rarely used constraints, missing dyads together with three of the constraints, and directed two-mode networks. Nor are there counterparts of the packages built on `ergm`: `tergm` for dynamic networks, `ergm.count` for counts and `ergm.multi` for several networks. For models that `nwergm` lacks, `ergm` remains the reference and the more complete program. The manual (Grund, 2026c) gives the status of every term.

11 Conclusions

`nwergm` brings exponential-family random graph models to Stata: 123 of the 139 terms of `ergm` 4.12.0 fully and two in part, four estimators with the defaults of `ergm`, constraints, missing-data handling, diagnostics, goodness of fit, information criteria, prediction and simulation, in a Monte Carlo sampler that runs as compiled code. On the same data, the estimates agree with those of `ergm` to within a tenth of a standard error. With the defaults of `ergm`, including its adaptive Monte Carlo sample size, the current version fits the eight benchmark models 2.1 to 4.1 times faster than `ergm`, with about as many

Monte Carlo proposals. Because an ERGM is fitted by simulation, an analysis should report the effective sample size, look at the trace plots and check the goodness of fit on features the model was not fitted to.

Also see

`nwcommands` (Grund, 2026a), which includes `nwplot` for plotting observed and simulated networks; further model packages and extensions: `https://nwcommands.org`.

Acknowledgments

`nwergm` would not exist without the statnet project. The `ergm` package (Hunter and Handcock, 2006; Handcock et al., 2008), developed by the statnet team and many contributors, is the reference implementation that `nwergm` follows and is compared with; its source code and documentation were the specification for the Stata code. I am grateful to the developers for making their software public.

Use of AI in developing the software

The software was developed with the help of an AI coding assistant (Claude Code, Anthropic), working under my direction. I specified the scope, reviewed the code, and am responsible for the package.

About the author

Thomas U. Grund is a sociologist at the Institute of Sociology, RWTH Aachen University, working on social network analysis and computational social science. He is the author of `nwcommands`, a Stata package for the analysis of network data, and of `nwergm`, which builds on it.

12 References

Batagelj, V., and A. Mrvar. 2001. A subquadratic triad census algorithm for large sparse networks with small maximum degree. *Social Networks* 23(3): 237–243.

Breiger, R. L. 1974. The Duality of Persons and Groups. *Social Forces* 53(2): 181.

Davis, A., B. B. Gardner, and M. R. Gardner. 2022. *Deep South.* University of Chicago Press. Reissue of the 1941 edition, with a foreword by Isabel Wilkerson.

van Duijn, M. A. J., K. J. Gile, and M. S. Handcock. 2009. A Framework for the Comparison of Maximum Pseudo-Likelihood and Maximum Likelihood Estimation of Exponential Family Random Graph Models. *Social Networks* 31(1): 52–62.

Firth, D. 1993. Bias Reduction of Maximum Likelihood Estimates. *Biometrika* 80(1): 27–38.

Geyer, C. J., and E. A. Thompson. 1992. Constrained Monte Carlo Maximum Likelihood for Dependent Data. *Journal of the Royal Statistical Society, Series B* 54(3): 657–683.

Grund, T. U. 2026a. nwcommands: Scalable and Extensible Network Analysis in Stata. Manuscript submitted to the Stata Journal.

———. 2026b. nwplot: Flexible, Publication-Quality Network Visualization in Stata. Manuscript submitted to the Stata Journal.

———. 2026c. nwergm: Exponential Random Graph Models in Stata. Manual. Zenodo. `https://doi.org/10.5281/zenodo.23661331`.

Handcock, M. S., and K. J. Gile. 2010. Modeling Social Networks from Sampled Data. *The Annals of Applied Statistics* 4(1): 5–25.

Handcock, M. S., D. R. Hunter, C. T. Butts, S. M. Goodreau, and M. Morris. 2008. statnet: Software Tools for the Representation, Visualization, Analysis and Simulation of Network Data. *Journal of Statistical Software* 24(1): 1–11.

Hummel, R. M., D. R. Hunter, and M. S. Handcock. 2012. Improving Simulation-Based Algorithms for Fitting ERGMs. *Journal of Computational and Graphical Statistics* 21(4): 920–939.

Hunter, D. R. 2007. Curved Exponential Family Models for Social Networks. *Social Networks* 29(2): 216–230.

Hunter, D. R., S. M. Goodreau, and M. S. Handcock. 2008a. Goodness of Fit of Social Network Models. *Journal of the American Statistical Association* 103(481): 248–258.

Hunter, D. R., and M. S. Handcock. 2006. Inference in Curved Exponential Family Models for Networks. *Journal of Computational and Graphical Statistics* 15(3): 565–583.

Hunter, D. R., M. S. Handcock, C. T. Butts, S. M. Goodreau, and M. Morris. 2008b. ergm: A Package to Fit, Simulate and Diagnose Exponential-Family Models for Networks. *Journal of Statistical Software* 24(3).

Krivitsky, P. N. 2017. Using Contrastive Divergence to Seed Monte Carlo MLE for Exponential-Family Random Graph Models. *Computational Statistics & Data Analysis* 107: 149–161.

Krivitsky, P. N., D. R. Hunter, M. Morris, and C. Klumb. 2023. ergm 4: New Features for Analyzing Exponential-Family Random Graph Models. *Journal of Statistical Software* 105(6).

Lazega, E., and M. van Duijn. 1997. Position in Formal Structure, Personal Characteristics and Choices of Advisors in a Law Firm: A Logistic Regression Model for Dyadic Network Data. *Social Networks* 19(4): 375–397.

Lusher, D., J. Koskinen, and G. Robins, eds. 2013. *Exponential Random Graph Models for Social Networks: Theory, Methods, and Applications*. New York: Cambridge University Press.

Morris, M., M. S. Handcock, and D. R. Hunter. 2008. Specification of Exponential-Family Random Graph Models: Terms and Computational Aspects. *Journal of Statistical Software* 24(4): 1–24.

Padgett, J. F., and C. K. Ansell. 1993. Robust Action and the Rise of the Medici, 1400–1434. *American Journal of Sociology* 98(6): 1259–1319.

Snijders, T. A. B. 2002. Markov chain Monte Carlo estimation of exponential random graph models. *Journal of Social Structure* 3(2): 1–40.

Wang, P., G. Robins, and P. Pattison. 2009. *PNet: Program for the Simulation and Estimation of Exponential Random Graph (p^*) Models*. Melbourne School of Psychological Sciences, University of Melbourne.